

NLP Project - Report: Text Processing and Analysis of BBC News Dataset

Course: Natural Language Processing

1. Introduction & Dataset Description

This project aims to apply fundamental Natural Language Processing (NLP) techniques to real-world text data. The dataset selected for this project is the BBC News Dataset, which contains articles categorized into various topics.

For the scope of this project, we successfully extracted a balanced subset of 120 articles across three categories: **business** (40 articles), **politics** (40 articles), and **sport** (40 articles). This subset guarantees a massive corpus that exceeds the 5,000-word requirement, providing a rich vocabulary for text wrangling, chunking, and named entity recognition.

2. Methodology

The project was developed using Python and executed on Google Colab. We heavily relied on the **NLTK (Natural Language Toolkit)** library for core NLP tasks. Other libraries included **BeautifulSoup** for HTML parsing, **pandas** for data structuring, **matplotlib** for data visualization, and **ipywidgets** to build the interactive mini-application.

3. Results & Discussion

1. Task 1: Text Wrangling Pipeline

The text preprocessing pipeline involved several crucial steps to clean the raw text. We removed HTML tags, URLs, numbers, and special characters, retaining only English letters and basic punctuation. Then, the text was tokenized into sentences and words.

During the removal of stop words, we explicitly kept negation words (**not**, **no**, **nor**) because they carry significant semantic value and can drastically change the context of a sentence.

- **Discussion: Stemming vs. Lemmatization**

We applied both the Porter Stemmer and WordNet Lemmatizer to compare their outputs.

```
[92]
✓ Os
# 5- Comparison Table
# =====
sample_words = [
    "studies", "studying", "running",
    "better", "companies", "played",
    "playing", "political", "universities"
]

comparison_table = pd.DataFrame({
    "Original Word": sample_words,
    "Porter Stemmer": [stemmer.stem(word) for word in sample_words],
    "WordNet Lemmatizer": [lemmatizer.lemmatize(word) for word in sample_words]
})

comparison_table
```

	Original Word	Porter Stemmer	WordNet Lemmatizer
0	studies	studi	study
1	studying	studi	studying
2	running	run	running
3	better	better	better
4	companies	compani	company
5	played	play	played
6	playing	play	playing
7	political	polit	political
8	universities	univers	university

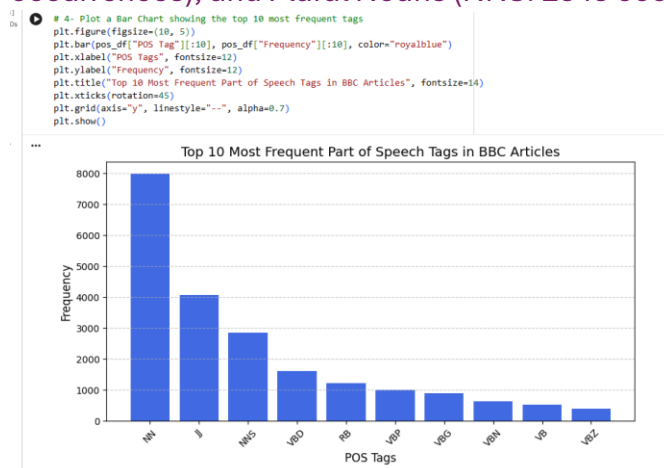
Next steps: [Generate code with comparison_table](#) [New interactive sheet](#)

• Analysis:

We observed that the Porter Stemmer can sometimes produce an incomplete or linguistically incorrect root. For example, the word "**universities**" becomes "**univers**", which is an example of over-stemming. In contrast, the WordNet Lemmatizer returns "**university**", which is the correct base word. Another example is "**studies**": the Stemmer returns "**studi**", while the Lemmatizer returns "**study**". This shows that lemmatization usually produces more meaningful English words, whereas stemming mainly truncates word endings.

2. Task 2: Part-of-Speech (POS) Tagging

We applied NLTK's averaged perceptron tagger to determine the grammatical parts of speech for all filtered words across the 120 articles. The frequency analysis revealed that the top three most frequent tags were Singular Nouns (NN: 7976 occurrences), Adjectives (JJ: 4061 occurrences), and Plural Nouns (NNS: 2845 occurrences).



3. Task 3: Chunking and Chinking

To extract Noun Phrases (NP), we designed a specific regular expression grammar.

- **Chunk Rule: {<DT>?<JJ>*<NN.*>+}** to group an optional determiner, adjectives, and nouns.
- **Chink Rule: }<VB.*|IN>+{** to strictly exclude any verbs or prepositions that might accidentally be included in the phrase.

```

--- Displaying Chunk Trees for Example Sentences ---

Sentence (1): Crucial decision on super casinos A decision on whether to allow westminster to legislate on super casinos is set to be made by the Scottish Parliament.

Chunk Tree (Bracketed Notation):
(S
  (NP Crucial/JJ decision/NN)
  (S
    (NP super/JJ casinos/NNS)
    (NP A/DT decision/NN)
    (VP
      (VBZ will/MD)
      (VP
        (NP westminster/NRP)
        (VP
          (VBZ legislate/VB)
          (NP super/JJ casinos/NNS)
          (PP on/IN)
          (NP whether/IN)
          (NP A/DT super/NN casino/NN)
          (NP will/MD)
          (NP be/VB)
          (NP will/VBN)
          (NP the/DT Scottish/JJ Parliament/NRP)
          (./.))
        (./.))
      (./.))
    (./.))
  (./.))

Sentence (2): The government has plans for up to eight Las Vegas style/NB resorts in the UK one of which is likely to be in Glasgow.

Chunk Tree (Bracketed Notation):
(S
  (NP The/DT government/NR)
  (VP
    (VBZ has/VBZ)
    (NP plans/NNS)
    (PP for/IN)
    (NP
      (NP up/TO)
      (NP to/VB)
      (NP eight/CD)
      (NP Las/NRP Vegas/NRP style/NB resorts/NNS)
      (PP in/IN)
      (NP the/DT UK/NRP)
      (PP one/CD)
      (NP of/IN)
      (NP which/WH)
      (NP is/VBZ)
      (NP likely/ADJ)
      (NP to/VB)
      (NP be/VB)
      (NP in/IN)
      (NP Glasgow/NRP)
      (./.))
      (./.))
    (./.))
  (./.))

Sentence (3): Scottish ministers insist they will still have the final say on whether a super casino will be built in Scotland.

Chunk Tree (Bracketed Notation):
(S
  (NP Scottish/JJ ministers/NNS)
  (VP
    (VBZ insist/VBP)
    (NP they/PRP)
    (VP
      (VBZ will/MD)
      (VP
        (NP still/AD)
        (VP
          (VBZ have/VB)
          (NP the/DT final/JJ say/NN)
          (PP on/IN)
          (NP whether/IN)
          (NP A/DT super/NN casino/NN)
          (NP will/MD)
          (NP be/VB)
          (NP will/VBN)
          (NP in/IN)
          (NP Scotland/NRP)
          (./.))
          (./.))
        (./.))
      (./.))
    (./.))
  (./.))

Sentence (4): But opposition parties say that will not happen in practice.

Chunk Tree (Bracketed Notation):
(S
  (NP But/CC)
  (NP opposition/NN parties/NNS)
  (VP
    (VBZ say/VBP)
    (NP that/DT)
    (VP
      (VBZ will/MD)
      (VP
        (NP not/AD)
        (VP
          (VBZ happen/VB)
          (PP in/IN)
          (NP practice/NN)
          (./.))
          (./.))
        (./.))
      (./.))
    (./.))
  (./.))

```

4. Task 4: Named Entity Recognition (NER)

Using NLTK's **ne_chunk**, we extracted named entities from the dataset. The model successfully identified thousands of entities, primarily classified as PERSON (1101), ORGANIZATION (814), and Geopolitical Entities GPE (804). However, during our manual error analysis, we spotted some interesting misclassifications:

```

from nltk.chunk import ne_chunk
from nltk.tag import pos_tag
from nltk.tokenize import word_tokenize

for ent_type, count in type_counts.items():
    print(f'{ent_type}: {count}')

print("\n--- Sample of Extracted Entities for Error Analysis ---")
print(all_entities[:100])

---
--- Entity Type Statistics ---
GPE: 804
ORGANIZATION: 814
PERSON: 1101
FACILITY: 18
LOCATION: 4
GSP: 41

--- Sample of Extracted Entities for Error Analysis ---
[('Crucial', 'GPE'), ('Scottish Parliament', 'ORGANIZATION'), ('Glasgow', 'GPE'), ('Scotland', 'GPE'), ('Scottish Executive', 'ORGANIZATION'), ('Glasgow',

```

Discussion of Ambiguous or Misclassified Entities:

1. First Entity ("Jack Straw"):

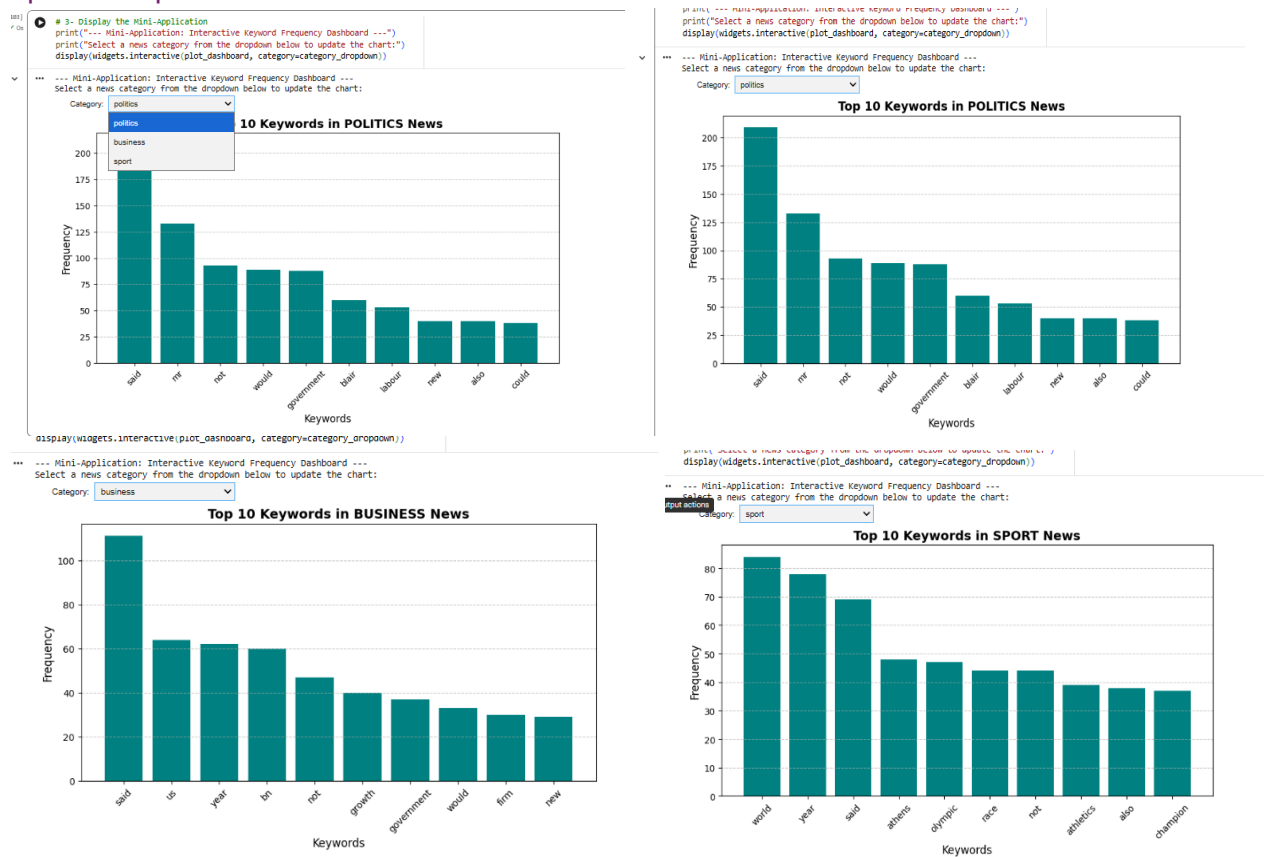
The entity "Jack Straw" was misclassified by the NLTK model as a **FACILITY**. In reality, Jack Straw is a person (a British politician). This ambiguity likely occurred due to the surrounding context in the news article (e.g., referring to "Jack Straw's country residence"), which confused the chunker into treating his name as the title of a building or a physical establishment.

2. Second Entity ("Madrid" / "Gibraltar"):

The entities "Madrid" and "Gibraltar" were completely misclassified as **PERSON**. These are well-known geographical locations and should be classified as Geopolitical Entities (GPE). This error happens because NLTK's built-in chunker relies heavily on capitalization and immediate surrounding words. In certain sentence structures, the model misinterprets the place name acting as a subject for a human being.

5. Task 5: Mini-Application (Interactive Keyword Dashboard)

To demonstrate a practical application of our cleaned text data, we developed an **Interactive Keyword Frequency Dashboard**. This tool allows users to select a specific news category (e.g., Politics, Business, or Sport) via a dropdown menu. The application dynamically filters the dataset and renders a bar chart displaying the top 10 most frequent keywords for that specific topic.



This mini-app can be highly useful for journalists or media analysts who want to quickly grasp the main topics or trends currently dominating a specific news category without reading hundreds of articles.

4. Conclusion

This project successfully demonstrated the end-to-end process of Natural Language Processing. From rigorous data cleaning and tokenization to advanced techniques like Chunking and Named Entity Recognition, we transformed unstructured raw text into structured, analyzable data. The insights gained from the POS distribution and the interactive dashboard highlight the power of NLP in summarizing and understanding large corpora of text.